

Week2-Exercise2.2_Hembree, A

June 21, 2026

0.0.1 DSC 670 - Advanced Uses of Generative AI

0.0.2 Week 2, Exercise 2.2

0.0.3 Hembree, Lex

Problems 1-3 Assignment Instructions There are several LLM prompt engineering problems below. Your assignment is to write one or more prompts for each problem using any of the techniques you learned this week.

For problems #1-3, you can use any LLM interface (e.g., ChatGPT, Google Gemini, CoPilot). Note that problem #4 requires the use of the OpenAI API. Write a 250–500-word formal paper on the results for problems #1-3. Your paper must follow this format provided in the APA Help Center. Problem #4 has specific instructions directly below its description.

0.1 Prompt Engineering Write-Up

This paper documents the results produced by the crafting of a prompt using three LLM interfaces to complete text generation, sentiment analysis, and mathematical reasoning tasks. Each problem had a set of rules to follow and required different prompts to return the results below.

0.1.1 Problem 1: Text Generation

Prompt Used

Using APA-compliant references and an active voice; provide me a summary on prompt engineering that is exactly 200 words #### LLM Interface Output (Copilot) Prompt engineering is the process of designing inputs that guide a large language model toward a specific outcome. I shape the model's behavior by giving clear instructions, defining the task, and setting boundaries that reduce ambiguity. When I write a prompt, I focus on intent, structure, and context because the model responds directly to the information I provide. I treat the prompt as an operational directive that tells the system what to generate, how to format it, and what voice to use. I improve prompt performance by breaking complex tasks into smaller parts, adding examples, and specifying constraints such as length or perspective. I also refine prompts through iteration. Each revision helps me identify gaps, misunderstandings, or missing details that affect the output. I adjust wording, add context, or tighten requirements until the model produces consistent results. Prompt engineering matters because the model does not infer my goals unless I state them. The quality of the output depends on how precisely I communicate expectations. When I apply structured prompting techniques, I

increase accuracy, reduce errors, and produce outputs that align with academic, technical, or professional needs. ##### Reference Reynolds, L., & McDonell, K. (2021). Prompt programming for large language models: Beyond the few-shot paradigm. arXiv. <https://arxiv.org/abs/2102.07350> ##### Analysis Utilizing prior work experience, I crafted a prompt that outlined the specifications of the problem. While the format and content provided fits what I asked for, the length is incorrect. The interface provided only 186 out of 200. Even if I include the reference at 19 words, the response would be over the limit. The limiting factor for the interface is that it typically cannot give exact word counts but rather will get close to the request and ask the user if they want to lengthen or shorten the response. This output was close enough that it could be expanded upon and tweaked until it fits the required word count.

0.1.2 Problem 2: Sentiment Analysis

Prompt Used

Provide a sentiment analysis on the following input and provide me an output that equals the emotions the text conveys. input: That was a FINE example of teaching an intelligent automation class. :-/ ##### LLM Interface Output (Gemini) Sentiment Analysis Primary Sentiment: Sarcastic / Frustrated Tone: Passive-aggressive, disappointed, and annoyed. Context: The word “FINE” in all caps, paired with the irritated/skeptical emoticon :-/, heavily implies sarcasm. The user is expressing that the class was actually not fine, likely suffering from poor quality, irony (given it’s an “intelligent automation” class that perhaps lacked intelligence or automation), or severe mismanagement. Empathic Echo Output “Wow. That was just brilliant. Absolutely seamless. Because nothing says ‘cutting-edge intelligent automation’ quite like a chaotic mess that makes you question the very definition of intelligence. Just spectacular. :-/” ##### Analysis Gemini surprised me here by picking up on the sarcasm almost immediately. The output was almost identical to what my own interpretation of the required input was. It identified the tone, the emotional cues, and the intent behind the capitalization and the emoticon without needing extra context. That level of alignment shows that the model is pulling from a broad range of conversational patterns and has strong pattern-matching for tone markers. With Gemini being newer than other interfaces, it may have access to larger or more current datasets and token distributions. That could give it an advantage when interpreting informal language, sarcasm, or mixed emotional signals. This type of performance makes it a strong candidate for conversational apps and products where tone detection and quick inference matter. It handled the sentiment task like a system designed for dialogue rather than a system optimized only for structured outputs. (This one may turn into Skynet prof.)

0.1.3 Problem 3: Simple Math

Prompt Used

Returning only the result in a fraction format, what is the sum of 2,292.32 and 4494 1/3? ##### LLM Interface Output (ChatGPT) 508999/75 ##### Analysis Based on the problem parameters, I crafted the prompt to return only the answer to the math problem. Due to the prompt being explicit, narrow, and leaving no room for interpretation; it did exactly as prompted and provided no other responses, explanations,

or future actions. This behavior demonstrates how a tightly scoped prompt creates a clear operational boundary for the model. When the end state is explicit, the model treats the instruction as a hard stop and avoids generating extra material that would violate the constraints. This outcome reinforces a key principle of prompt engineering: specificity controls output. When the prompt defines the format, the content, and the stopping point, the model reliably conforms to those limits.

Problem #4 - Extraction Using the OpenAI API, extract the text from this invoice and then use an LLM to convert the text to structured JSON (you can literally ask the LLM to produce JSON).

Build the solution using a Jupyter Notebook (no other Python is acceptable) and include Markdown that thoroughly explains your thought process and your commentary on the results you achieved. Obvious Markdown that explicitly only explains the code will be ignored, since I can read the code. I'm looking for your commentary and evaluation in your own words.

After running into paywall issues with OpenAI API, I used BERT (Bidirectional Encoder Representations from Transformers) which is a Natural Language Processing (NLP) model developed by Google. It can be used for translation, text generation, and text summarization. Medium.com classifies it as a LLM so it fits into the parameters of the assignment, most at least since we are supposed to use OpenAI API. The objective was to pull text from the invoice and convert it into structured JSON.

```
[2]: # Import library
import pdfplumber

# Read the invoice
invoice_path = "C:/Users/Ashem/OneDrive/Desktop/Masters Program - DSC/DSC 670 - Advanced Uses of Generative AI/Week 2/dsc-670-exercise-invoice.pdf"

# Extract the text
with pdfplumber.open(invoice_path) as pdf:
    extracted_text = "\n".join(page.extract_text() for page in pdf.pages)

# ensure it read
print(extracted_text)
```

INVOICE

Company Name

321 Avenue A Date: 6/28/2024

Portland, OR 12345 Invoice # 1111

Phone: (206) 555-1163 For PO # 123456

Fax: (206) 555-1164

someone@websitegoeshere.com

Bill To:

Natasha Jones

Items over this amount qualify for an additional discount \$100

Central Beauty

123 Main St.

% discount 10%

Manhattan, NY 98765

(321) 555-1234

Quantity	Description	Unit price	Amount	Discount applied
1	Item Number 1	\$ 2 .00	\$ 2 .00	
1	Item Number 2	\$ 2 .00	\$ 2 .00	
1	Item Number 3	\$ 2 .00	\$ 2 .00	
	\$ -			
	\$ -			
	\$ -			
	\$ -			
	\$ -			
	\$ -			
	\$ -			
	\$ -			
	\$ -			
	Subtotal	\$ 6 .00		

Make all checks payable to <Company name>. Credit \$ 1,000.00

If you have any questions concerning this invoice, contact Tax 9.80%

<Name> at <phone or email>.

Additional discount 12%

Thank you for your business! Balance due \$ (994.20)

I used pdfplumber for extraction because it gives direct access to page text. The extraction step produced a clean text block with all invoice fields visible. I used that model to identify entities in the extracted text. The model tagged locations, organizations, and person names. I need the model to be able to read the raw text it extracts so I need to run it through BERT so it gets a structured format it can do that with. (I'd do way more Bert and Ernie jokes but I can't remember most of sesame street at this point)

```
[3]: # Import library
from transformers import AutoTokenizer, AutoModelForTokenClassification
import torch

# Load BERT from local area
tokenizer = AutoTokenizer.from_pretrained("dslim/bert-base-NER")
model = AutoModelForTokenClassification.from_pretrained("dslim/bert-base-NER")
```

Some weights of the model checkpoint at dslim/bert-base-NER were not used when initializing BertForTokenClassification: ['bert.pooler.dense.bias', 'bert.pooler.dense.weight']

- This IS expected if you are initializing BertForTokenClassification from the checkpoint of a model trained on another task or with another architecture (e.g. initializing a BertForSequenceClassification model from a BertForPreTraining model).
- This IS NOT expected if you are initializing BertForTokenClassification from the checkpoint of a model that you expect to be exactly identical (initializing a BertForSequenceClassification model from a BertForSequenceClassification model).

```
[4]: # Run BERT (Ernie is missing)
inputs = tokenizer(extracted_text, return_tensors="pt", truncation=True)
outputs = model(**inputs)
preds = torch.argmax(outputs.logits, dim=2)
tokens = inputs.tokens()
labels = [model.config.id2label[p.item()] for p in preds[0]]
```

After never using BERT before and getting through with minimal network issues (own system errors) that prevented the code from running, the system ran without error and allowed the JSON code to write an output.

```
[5]: # Import library
import json

# Convert to JSON
entities = []
for token, label in zip(tokens, labels):
    if label != "0":
        entities.append({"token": token, "label": label})

json_output = json.dumps({"entities": entities}, indent=4)
print(json_output);
```

```
{
  "entities": [
    {
      "token": "Portland",
      "label": "B-LOC"
    },
    {
      "token": "OR",
      "label": "B-LOC"
    },
    {
      "token": "Natasha",
      "label": "B-PER"
    },
    {
      "token": "Jones",
      "label": "I-PER"
    },
    {
      "token": "Central",
      "label": "B-ORG"
    },
    {
      "token": "Beauty",
      "label": "I-ORG"
    }
  ]
}
```

```

    },
    {
      "token": "Main",
      "label": "B-LOC"
    },
    {
      "token": "St",
      "label": "I-LOC"
    },
    {
      "token": "Manhattan",
      "label": "B-LOC"
    },
    {
      "token": "NY",
      "label": "B-LOC"
    }
  ]
}

```

The BERT model performed as expected for a general Named Entity Recognition (NER) model, it identified names and locations correctly. What it did not identify, were invoice-specific fields such as totals, dates, or line items because the model is not trained for financial document structure. This is a limitation of the model rather than the extraction process; this is the shortfall I would need to determine a path forward in dealing with during other applications of this output. Otherwise the extracted text was complete, with the bottleneck unfortunately being the model's domain knowledge.

The JSON output reflects the entities the model was able to detect. The structure is valid and consistent. The output is not a full invoice schema, but it is a structured representation of the model's interpretation of the text. This aligns with the assignment requirement to convert extracted text into structured JSON using an LLM.

[]:

Prompt Engineering Write-Up

This paper documents the results produced by the crafting of a prompt using three LLM interfaces to complete text generation, sentiment analysis, and mathematical reasoning tasks. Each problem had a set of rules to follow and required different prompts to return the results below.

Problem 1: Text Generation

Prompt Used

Using APA-compliant references and an active voice; provide me a summary on prompt engineering that is exactly 200 words

LLM Interface Output (Copilot)

Prompt engineering is the process of designing inputs that guide a large language model toward a specific outcome. I shape the model's behavior by giving clear instructions, defining the task, and setting boundaries that reduce ambiguity. When I write a prompt, I focus on intent, structure, and context because the model responds directly to the information I provide. I treat the prompt as an operational directive that tells the system what to generate, how to format it, and what voice to use.

I improve prompt performance by breaking complex tasks into smaller parts, adding examples, and specifying constraints such as length or perspective. I also refine prompts through iteration. Each revision helps me identify gaps, misunderstandings, or missing details that affect the output. I adjust wording, add context, or tighten requirements until the model produces consistent results.

Prompt engineering matters because the model does not infer my goals unless I state them. The quality of the output depends on how precisely I communicate expectations. When I apply structured prompting techniques, I increase accuracy, reduce errors, and produce outputs that align with academic, technical, or professional needs.

Reference

Reynolds, L., & McDonell, K. (2021). Prompt programming for large language models: Beyond the few-shot paradigm. arXiv. <https://arxiv.org/abs/2102.07350>

Analysis

Utilizing prior work experience, I crafted a prompt that outlined the specifications of the problem. While the format and content provided fits what I asked for, the length is incorrect. The interface provided only 186 out of 200. Even if I include the reference at 19 words, the response would be over the limit. The limiting factor for the interface is that it typically cannot give exact

word counts but rather will get close to the request and ask the user if they want to lengthen or shorten the response. This output was close enough that it could be expanded upon and tweaked until it fits the required word count.

Problem 2: Sentiment Analysis

Prompt Used

Provide a sentiment analysis on the following input and provide me an output that equals the emotions the text conveys. input: That was a FINE example of teaching an intelligent automation class. :-/

LLM Interface Output (Gemini)

Sentiment Analysis Primary Sentiment: Sarcastic / Frustrated Tone: Passive-aggressive, disappointed, and annoyed. Context: The word "FINE" in all caps, paired with the irritated/skeptical emoticon :-/, heavily implies sarcasm. The user is expressing that the class was actually not fine, likely suffering from poor quality, irony (given it's an "intelligent automation" class that perhaps lacked intelligence or automation), or severe mismanagement.

Empathic Echo Output "Wow. That was just brilliant. Absolutely seamless. Because nothing says 'cutting-edge intelligent automation' quite like a chaotic mess that makes you question the very definition of intelligence. Just spectacular. :-/"

Analysis

Gemini surprised me here by picking up on the sarcasm almost immediately. The output was almost identical to what my own interpretation of the required input was. It identified the tone, the emotional cues, and the intent behind the capitalization and the emoticon without needing extra context. That level of alignment shows that the model is pulling from a broad range of conversational patterns and has strong pattern-matching for tone markers.

With Gemini being newer than other interfaces, it may have access to larger or more current datasets and token distributions. That could give it an advantage when interpreting informal language, sarcasm, or mixed emotional signals. This type of performance makes it a strong candidate for conversational apps and products where tone detection and quick inference matter. It handled the sentiment task like a system designed for dialogue rather than a system optimized only for structured outputs. (This one may turn into Skynet prof.)

Problem 3: Simple Math

Prompt Used

Returning only the result in a fraction format, what is the sum of 2,292.32 and 4494 1/3?

LLM Interface Output (ChatGPT)

508999/75

Analysis

Based on the problem parameters, I crafted the prompt to return only the answer to the math problem. Due to the prompt being explicit, narrow, and leaving no room for interpretation; it did exactly as prompted and provided no other responses, explanations, or future actions.

This behavior demonstrates how a tightly scoped prompt creates a clear operational boundary for the model. When the end state is explicit, the model treats the instruction as a hard stop and avoids generating extra material that would violate the constraints. This outcome reinforces a key principle of prompt engineering: specificity controls output. When the prompt defines the format, the content, and the stopping point, the model reliably conforms to those limits.